AI Agents, Tools and Evaluation

Generative AI handbook, volume 3 · Version 2026.2

> Sample foundation document written for testing a domain-expert RAG. It summarises widely used concepts; check vendor documentation for current limits and product details.

1. From chatbots to agents

An AI agent is a system in which a language model decides what to do next, uses tools to act, observes the results and repeats until the goal is met. A chatbot answers once; an agent runs a loop. Agents are useful for open-ended tasks with several steps, such as investigating an incident, changing code across files or reconciling records, where the steps cannot be fixed in advance.

2. The agent loop

1. **Goal:** receive a task and any constraints.
2. **Think:** decide the next step from the goal and what has happened so far.
3. **Act:** call a tool with arguments (search, database query, code execution, HTTP request).
4. **Observe:** add the tool result to the context.
5. **Stop or continue:** finish when the goal is met, a step or cost limit is reached, or a human must approve.

This interleaving of reasoning and acting is often called ReAct. Variants add an explicit plan first (plan-and-execute) or a self-critique step after each result (reflection).

3. Tool calling

Models call tools by producing structured output that matches a JSON schema supplied by the application. The application, not the model, executes the tool and returns the result. Good tool design matters more than prompt wording:

- Give each tool one clear purpose, a descriptive name and a description of when to use it.
- Use typed parameters with enums and ranges instead of free text.
- Return concise, structured results; truncate large outputs and say so.

- Return helpful error messages the model can act on.
- Make side-effecting tools idempotent and require confirmation for irreversible actions.

```
{"name": "search_invoices",
 "description": "Find invoices by vendor and month. Use before answering any ques
 "parameters": {"type": "object",
   "properties": {"vendor": {"type": "string"}, "month": {"type": "string", "patt
   "required": ["vendor", "month"]}}
```

4. Workflow patterns

Pattern	Description	Use when
Prompt chaining	Fixed sequence of model calls.	Steps are known in advance.
Routing	A classifier sends the input to a specialised path.	Distinct request types.
Parallelisation	Several calls at once, results merged or voted.	Independent sub-tasks or higher confidence.
Orchestrator-workers	A lead model delegates sub-tasks to worker calls.	Large tasks that split naturally.
Evaluator-optimiser	One call produces, another critiques, repeat.	Clear quality criteria.
Autonomous agent	The model chooses steps and tools in a loop.	Open-ended tasks where steps cannot be predicted.

Start with the simplest pattern that works. Autonomy adds cost, latency and new failure modes; add it only when fixed workflows fall short.

5. Memory

Short-term memory is the conversation and tool results in the context window. **Long-term memory** stores facts, preferences or past task outcomes outside the model, usually in a database or vector store, and retrieves them when relevant. Summarise long histories to stay within the context window, and never store secrets or personal data in memory without the same controls as any other database.

6. Model Context Protocol

The Model Context Protocol (MCP) is an open standard for connecting AI applications to tools and data. A host application runs MCP clients that connect to MCP servers; servers expose tools, resources and prompts over JSON-RPC, using stdio for local servers or Streamable HTTP for remote ones. MCP standardises the connection, not the trust: tool descriptions and results reach the model as text, so a malicious server can attempt prompt injection. Install servers from trusted sources, scope their credentials to the user, and show tool calls to the user before side effects.

7. Multi-agent systems

Multi-agent designs split work between specialised agents, for example a planner, a researcher and a reviewer. They help when sub-tasks need different tools or instructions, or when an independent reviewer improves quality. They also multiply cost and make failures harder to trace. Log every message between agents and give each agent the minimum tools it needs.

8. Evaluating LLM applications

Method	Strength	Watch out for
Golden test sets	Repeatable, catches regressions	Must be refreshed as the product changes
Deterministic checks	Cheap and exact (JSON validity, required strings, citation IDs)	Only cover what can be expressed as rules
LLM-as-judge	Scales to open-ended quality	Position bias, verbosity bias, self-preference; calibrate against human labels
Human review	Ground truth for nuanced quality	Slow and expensive; sample strategically
Online metrics	Real user outcomes: thumbs, task completion, escalations	Noisy; needs enough traffic

For agents, evaluate the trajectory as well as the final answer: did it choose the right tools, in a sensible order, within the step budget, without unsafe actions?

9. Cost and latency

- Cache repeated prompts and prefixes; many APIs discount cached input tokens.
- Route easy requests to smaller models and hard ones to larger models.
- Stream output so users see progress.
- Trim context: fewer, better passages beat many mediocre ones.
- Batch offline work, such as document enrichment, through batch APIs.
- Set max_tokens, step limits and per-user budgets to prevent runaway cost.

10. LLMOps checklist

Area	Practice
Versioning	Version prompts, models, retrieval settings and datasets together.
Testing	Golden set and safety tests in CI; block releases on regressions.
Observability	Trace each request: prompt, retrieved passages, tool calls, tokens, latency, cost.
Feedback	Collect user feedback and route failures into the test set.
Governance	Record the model, data sources and owner of every AI feature.